

CS 3081 · ARTIFICIAL INTELLIGENCE · FALL 2026

Assignment 1 — Search Strategies

BFS · DFS · Greedy Best-First · A*

Dr Naila Marir

Instructor: Dr Naila Marir**Sections:** 3081-1 (CRN 10480), 3081-2 (CRN 10481)**Total marks:** 100

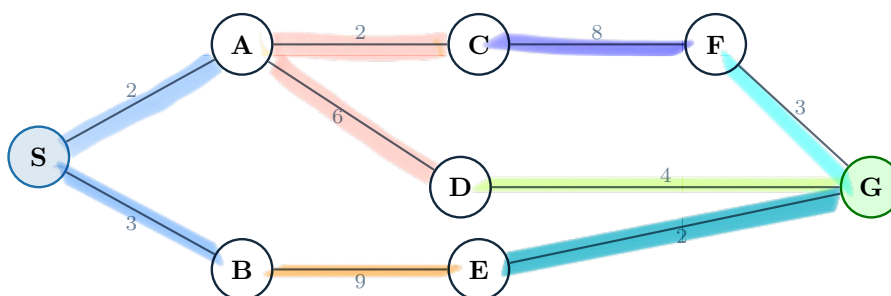
Remas Pawas S24109161

Instructions. Answer all four questions. Show your working — a correct final answer with no reasoning earns partial credit only. Where a question asks for the order in which nodes are expanded, write the nodes as a sequence, e.g. S, A, C, G .

- When two nodes have equal priority, break ties **alphabetically**.
- A node is *expanded* when it is removed from the frontier and its successors are generated. Counting the goal test, a node is expanded even if it turns out to be the goal.
- Assume the **graph-search** form of each algorithm: never expand the same state twice.
- This is an individual assignment. You may discuss ideas, but the work you submit must be your own.

1 Uninformed search: BFS and DFS

Consider the following state graph. The start state is S and the goal state is G . Edges are undirected, and the number on each edge is its step cost.



For this question ignore the edge costs. BFS and DFS are *uninformed*: they do not look at cost or at any estimate of distance to the goal.

(a) Breadth-first search : First in, First out

[8 marks]

List the order in which states are expanded by BFS, and give the path it returns.

Answer: $S \rightarrow [A, B]$ $A \rightarrow [B, C, D]$ $B \rightarrow [C, D, E]$ $C \rightarrow [D, E, F]$ $D \rightarrow [E, F, G]$ $E \rightarrow [F, G]$	Expansion Order: S, A, B, C, D, E, F, G Returned Path: $S \rightarrow A \rightarrow D \rightarrow G$
--	--

(b) Depth-first search

[8 marks]

List the order in which states are expanded by DFS, and give the path it returns. Assume successors are pushed onto the stack in reverse alphabetical order, so that the alphabetically first successor is expanded next.

DFS : Last in, First Out

Answer:

$S \rightarrow [B, A]$
 $A \rightarrow [B, D, C]$
 $C \rightarrow [B, D, F]$
 $F \rightarrow [B, D, G]$

Expansion order: S, A, C, F, G
 returned path: S, A, C, F, G

(c) Comparison

[6 marks]

BFS and DFS returned different paths. State which path is shorter *in number of steps*, and explain why BFS is guaranteed to find that path while DFS is not.

Answer:

BFS : $S \rightarrow A \rightarrow D \rightarrow G$ 3 stepsDFS : $S \rightarrow A \rightarrow C \rightarrow F \rightarrow G$ 4 steps

* BFS; shorter and checks path level by level, so it finds the path with the fewest steps.

(d) Frontier data structures

[3 marks]

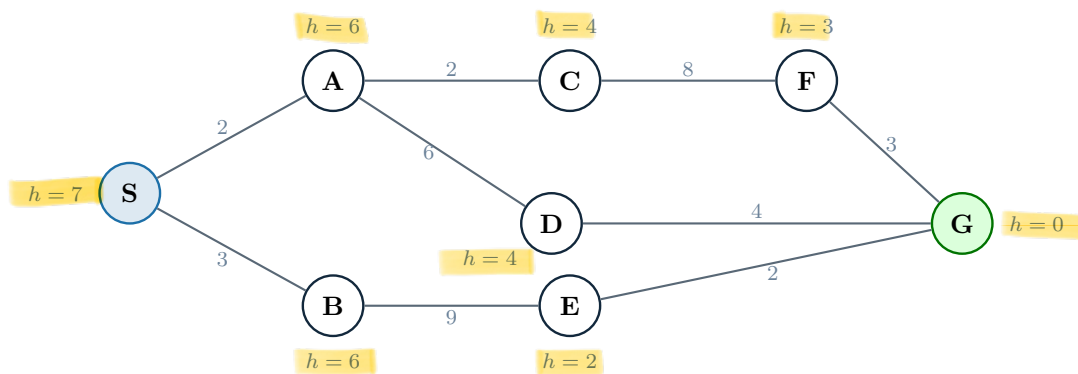
BFS and DFS differ by one design choice. Name the data structure each uses for its frontier, and explain in one sentence how that choice produces the difference in behaviour you observed.

Answer:

* BFS uses a Queue — FIFO it checks nodes level by level
 * DFS uses a Stack — FILO it follows one path deeply

2 Informed search: Greedy Best-First and A*: Choose the node with the Smallest h .

The same graph is repeated below. Each state is now annotated with a heuristic value $h(n)$ — an estimate of the cost from n to the goal G .



(a) Greedy best-first search

[8 marks]

Greedy best-first expands the frontier node with the smallest $h(n)$. List the order of expansion and the path returned.

Answer:

$S(7) \rightarrow [A(6), B(6)]$
 $A(6) \rightarrow [C(4), D(4), B(6)]$
 $C(4) \rightarrow [F(3), D(4), B(6)]$
 $F(3) \rightarrow [G(0), D(4), B(6)]$

Expansion Order: S, A, C, F, G
 Returned path: $S \rightarrow A \rightarrow C \rightarrow F \rightarrow G$

(b) A* search

[12 marks]

A* expands the frontier node with the smallest $f(n) = g(n) + h(n)$. Complete the table for every node *at the moment it is expanded*, then give the order of expansion and the path returned.

Order	Node	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
1	S	0	7	7
2	A	2	6	8
3	C	4	4	8
4	B	3	6	9
5	D	8	4	12
6	G	12	0	12

Order of expansion: S, A, C, B, D, G

Path returned: S → A → D → G Total cost: 12

(c) Why they differ

[6 marks]

Greedy best-first and A* returned different paths, or explored a different number of nodes, or both. Explain the difference by referring to what each algorithm ignores.

Answer: Greedy Best-First Search uses only $h(n)$ and ignores the path cost $g(n)$, so it returns the more expensive path. $S \rightarrow A \rightarrow C \rightarrow F \rightarrow G$ costing 15. A* uses both $g(n)$ and $h(n)$, so it returns the cheaper path $S \rightarrow A \rightarrow D \rightarrow G$ costing 12.

3 Admissibility and optimality

(a) Definition

[5 marks]

Define an **admissible heuristic**. Your definition must be precise enough that someone could check a given heuristic against it.

Answer: An admissible heuristic; never gives a value higher than the real cheapest cost to the goal.

(b) Testing the heuristic

[10 marks]

For the graph in Question 2, compute the true cost $h^*(n)$ of the cheapest path from each node to G, and decide whether the given h is admissible.

Node	S	A	B	C	D	E	F	G
$h(n)$ given	7	6	6	4	4	2	3	0
$h^*(n)$ true	12	10	11	11	4	2	3	0
$h \leq h^*$	yes	yes	yes	yes	yes	yes	yes	yes

Is h admissible? yes

If not, name every node that violates the condition: None

(c) Consequence

[6 marks]

Suppose a heuristic overestimates the true cost at exactly one node. Explain what this can do to the solution A^* returns, and why BFS could never suffer from this problem.

Answer: If the heuristic overestimates, A^* may choose the wrong path & return a more expensive solution. BFS is not affected because it does not use a heuristic.

(d) A trivial heuristic

[3 marks]

What does A^* reduce to when $h(n) = 0$ for every node? Justify your answer in one sentence.

Answer: when $h(n) = 0$, A^* becomes Uniform-Cost Search :
 $f(n) = g(n) + 0 = g(n)$

4 Comparing the four strategies

(a) Properties table

[8 marks]

Complete the table. Name the data structure each strategy uses for its frontier. For *optimal*, write Yes or No and state any condition required for that answer to hold.

Strategy	Frontier	Optimal?
Breadth-first	FIFO Queue	yes, if all step costs are equal
Depth-first	FIFO stacks	No
Greedy best-first	priority queue ordered by $h(n)$	No
A*	priority queue ordered by $f(n) = g(n) + h(n)$	yes, with an admissible heuristic

(b) Choosing a strategy

[13 marks]

For each scenario, name the most appropriate of the four strategies and justify your choice. Refer to the properties in part (a) — a bare name earns no marks.

- i. A robot must find *any* route out of a building during a fire. Memory is very limited; the route need not be shortest.

DFS, because it uses less memory and any route is acceptable

- ii. A route planner must return the *cheapest* driving route, and a good straight-line-distance estimate is available.

A^* , because it finds the cheapest route using the distance estimate

- iii. A puzzle has uniform step costs, no useful heuristic, and the solution is known to be shallow.

BFS, because costs are equal & the solution is shallow

(c) Short answer

[4 marks]

A^* is optimal but is often impractical on large problems. State the reason, and name one property of the search that makes it worse.

Answer: A^* uses a lot of memory because it stores many nodes
A large number of branches makes it worse

Submission. Submit a single PDF to Blackboard by the due date. Handwritten work is acceptable if scanned clearly. Include your name and student ID on the first page.

Q1: 25 Q2: 26 Q3: 24 Q4: 25 **Total: 100 marks**